36118 Applied Natural Language Processing

Master of Data Science and Innovation, UTS

Assessment Task 2B

End-to-End NLP Project — Technical Report

Sydney Liveability Explorer

Subject	36118 Applied Natural Language Processing
Submission	AT2 Part B — Group Technical Report
Due Date	Monday, 4 May 2026, 23:59
Group	Group 3, Autumn 2026
GitHub Repository	https://github.com/Nelkit/sydney-liveability-ai
Project Link	https://sydney-liveability-ai.vercel.app/

Team Members

Last Name	First Name	Student ID	GitHub Username
Chavez Calona	Nelkit	25609227	@Nelkit
Rodriguez Arciniegas	Juan David	25418551	@juandavidrodriguezar
Weng	Nian-Ya	25176165	@Amanda1005
Robinson	Luis Gerardo	25519000	@LuisRobinsonH
Srinivas	Padmasri	25536071	@Psri-01
Liao	Ying-Kai	25729500	@Ying-Kai-Liao

Contents

1 Executive Summary

Sydney Liveability AI addresses a practical and underserved need: helping newcomers to Sydney — international students, migrants, and young professionals — choose where to live based on evidence rather than anecdote. The project ingests six verified data sources: Reddit r/sydney posts (20,237 records across 563 suburbs via Arc Shift), City of Sydney community consultation PDFs (1,114 chunks across three reports), BOCSAR crime statistics (62 offence categories per suburb), TfNSW GTFS transport timetables (656 Greater Sydney suburbs), OpenStreetMap amenity data (ten POI categories via the Overpass API), and City of Sydney ArcGIS facility and walkability data. The NLP pipeline applies DeBERTa-v3-based aspect-level sentiment analysis across eight liveability dimensions, GoEmotions-based emotion profiling (DistilRoBERTa), and sentence embedding via all-MiniLM-L6-v2 to produce 72,276 ChromaDB chunks for semantic retrieval. A two-crew CrewAI architecture separates offline pre-computation (sentiment, GIS, crime, topic modelling) from a real-time query crew (router, crime, sentiment, GIS, comparator, and synthesiser agents) that answers natural-language questions using an LLM-agnostic model for response synthesis. The RAG evaluation harness ran 15 tier-stratified questions against the deployed system, recording a mean relevance score of 2.75 out of 3, a hallucination rate of 29.8% on demo-tier questions, and a 93.3% completion rate (14/15; one timeout due to an upstream LLM parsing failure). The application is deployed at https://sydney-liveability-ai.vercel.app/ and demonstrates a production-grade implementation of agentic RAG with transparent, auditable evidence traces surfaced in the frontend EvidenceDrawer.

---

2 Project Objectives and Scope

Sydney Liveability AI addresses a practical and underserved need: helping newcomers to Sydney, including international students, migrants, and young professionals, choose where to live based on evidence rather than anecdote. Existing tools such as Domain or Google Maps provide price and location data, but no tool synthesises what residents actually say about their neighbourhoods with objective civic data about safety, transport, and infrastructure in a queryable, plain-language interface.

The project builds a two-layer application. The first layer is an offline NLP pipeline that ingests six verified data sources, runs aspect-level sentiment analysis, generates sentence embeddings, and stores structured results in PostgreSQL and ChromaDB. The second layer is a real-time multi-agent reasoning system that answers natural language questions grounded in that pre-computed evidence, visualised through an interactive suburb-level dashboard.

In scope for this submission:

- Aspect-level sentiment analysis over Reddit r/sydney posts (40k+ records) using a transformer-based ABSA model across 8 liveability dimensions.
- Sentence embedding and semantic retrieval via ChromaDB (72,000+ indexed chunks).
- Multi-agent pipeline (CrewAI) with specialised agents for crime, sentiment, GIS, and comparison queries.
- Retrieval-Augmented Generation (RAG) using an LLM-agnostic model for response synthesis.

- Interactive Next.js dashboard with Leaflet map, per-suburb report views, and an evidence-traced chat interface.

Out of scope:

- Automated data refresh pipelines (ingestion is manual; data is a 2024/2026 snapshot).
- User authentication or personalised account storage.
- Suburb coverage beyond those with sufficient Reddit and civic data presence.

---

3 Project Phases and Team Contributions

The project was developed over five phases from data ingestion through to evaluation and reporting. Responsibilities were distributed across the team according to each member's technical focus area.

3.1 Phase Breakdown Table

Phase	Tasks	Team Member(s)
Phase 1: Data	Data acquisition and preprocessing from Reddit (Arc Shift), Community Insights PDF, BOCSAR CSV, TfNSW GTFS, OpenStreetMap (Overpass API), and City of Sydney ArcGIS.	All members
Phase 2: NLP Pipeline	Aspect-level sentiment analysis (DeBERTa-v3 ABSA), sentence embeddings (MiniLM), ChromaDB ingestion, spaCy preprocessing, emotion profiling (GoEmotions), coverage detection.	Juan David Rodriguez, Ying-Kai Liao
Phase 3: Multi-Agent Pipeline	CrewAI pipeline crew (pre-computation) and query crew (real-time), FastAPI backend, RAG with Open Router API, Synthesiser and Router agents.	All members
Phase 4: Frontend	Next.js dashboard, Leaflet map, aspect radar chart, emotion profile visualisation, conversational chat UI, EvidenceDrawer, ReportModal.	Nelkit Chavez, Ying-Kai Liao
Phase 5: Evaluation and Report	NLP metrics, RAG quality evaluation (LLM-judge harness), application evaluation, report, poster, peer review.	All members

3.2 Juan David Rodriguez

This team member contributed to the data ingestion and retrieval layers of the pipeline, with a focus on making community-sourced qualitative data accessible to the language model.
The primary contribution was the design and implementation of the full ingestion pipeline for the City of Sydney community consultation PDFs. Starting from structured JSON output produced by the PDF parser, each text chunk was embedded using the `all-MiniLM-L6-v2` sentence transformer model and written to the ChromaDB collection `sydney_liveability` with metadata including suburb,

theme, source document, page number, and chunk index. This pipeline processed 1,114 records across three official PDF reports, forming the qualitative backbone of the RAG system.
A secondary contribution was the completion of the Synthesiser agent's ChromaDB retrieval layer, which embeds each user query at runtime, retrieves the top five semantically relevant chunks, and injects them into the LLM prompt alongside structured PostgreSQL outputs and specialist agent results, enabling the model to cite resident quotes and PDF extracts by source and suburb.

3.3 Nelkit Chavez

This team member led the backend architecture, civic data pipeline, agent synthesis layer, and the frontend implementation.
Primary contributions included the design and implementation of the Synthesiser agent (`backend/agents/query/synthesiser.py`), which receives structured outputs from all specialist agents and composes a final markdown response using a scenario-aware prompt system distinguishing three query scenarios: `single`, `comparator`, and `out_of_scope`. The Synthesiser iterates all agent outputs to build the cited source list, injects a ChromaDB-retrieved context block before calling the synthesis LLM, and exposes a `SYNTHESIS_DEBUG_MODE` flag for selective output inspection.
Backend contributions also included the GIS agent data layer, the full FastAPI endpoint architecture (`/api/civic`, `/api/chat`, `/api/chat/stream`), and the in-memory caching system with double-checked locking that prevents connection pool exhaustion on Supabase by loading all suburb data once and applying weight changes in pure Python on subsequent requests. A standalone data contribution was the civic ingestion pipeline: facility data from the City of Sydney ArcGIS REST API and walkability scores from the WalkScore API were cleaned, spatially joined with suburb polygons, and ingested into the `suburbs` PostgreSQL table with PostGIS geometry, forming the primary data source consumed by the GIS agent.
On the frontend, this team member designed and built the three-column main layout, the `OnboardingPanel` conversational weight-setting flow, the `MapPanel` with Leaflet polygon rendering and `flyToBounds` behaviour, the `EvidenceDrawer` pipeline trace panel, the `ReportModal` for individual and comparative reports, and the application-wide OKLCH design system with Framer Motion transitions.

3.4 Nian-Ya Weng

This team member contributed to the crime data pipeline, covering data acquisition, cleaning, structured ingestion, and agent implementation.
The primary contribution was the identification and processing of suburb-level crime data from BOCSAR. A suburb-level dataset (`SuburbData25Q4.csv`) was sourced directly from the BOCSAR Open Datasets portal, filtered to the five MVP suburbs, and aggregated across the most recent 24 months (January 2024 to December 2025) into a single `incident_count` per suburb per crime category. The cleaned output (`data/processed/bocsar_clean.csv`) contains 310 rows across 62 offence categories per suburb.
A secondary contribution was the implementation of the ingestion script (`backend/scripts/ingest_bocsar.py`), which maps the cleaned CSV to the `Bocsar` ORM model and writes 310 rows into the PostgreSQL `bocsar` table. The Crime agent (`backend/agents/query/crime.py`) was also implemented, replacing the placeholder with real SQLAlchemy queries that return per-suburb crime summaries and year-on-year trend signals to the Synthesiser.

Exploratory data analysis is documented in `notebooks/01_bocsar_clean.ipynb` and `notebooks/01_eda_and_cleaning.ipynb`, including suburb-level visualisations.

3.5 Padmasri Srinivas

This team member contributed the transport data dimension end-to-end and led the Sprint 4 RAG evaluation pipeline.

The primary contribution was the design and implementation of `data_extraction/extract_transport.py`, a reproducible pipeline that derives a normalised peak-hour transport frequency score for 656 Greater Sydney suburbs from the TfNSW GTFS Timetables (Full) dataset. The pipeline spatially joins 48,195 GTFS stops against ABS SAL 2021 suburb polygons using `geopandas`, chunks through the 500 MB+ `stop_times.txt` file at 100,000 rows at a time, filters to weekday peak-hour departures (07:00–08:59), and aggregates to a per-suburb mean services-per-hour score normalised 0–1. The resulting `data/processed/transport_scores.json` was ingested into the `transport_scores` PostgreSQL table via `data_extraction/ingest_transport.py`, which extended the ORM model with two GTFS-specific columns via Alembic migration `20260428_0006` and performs idempotent upserts tagged `source='gtfs'`.

A secondary contribution was the Sprint 4 evaluation harness: `evaluation/evaluate_rag.py` runs 15 tier-stratified questions through `/api/chat`, records full response payloads, and embeds aggregated metrics (relevance, hallucination rate, blockquote compliance, latency) into a single UI-ready `data/eval/rag_evaluation.json`. The evaluation produced a mean relevance score of 2.75 and a hallucination rate of 29.8% across demo-tier questions, with qualitative findings on suburb-scoping bugs and Supabase session saturation that informed backend patches during the final sprint.

3.6 Ying-Kai Liao

This team member owned the Reddit data layer, the offline NLP pipeline, and the agentic RAG rework of the query crew.

The Reddit contribution pivoted extraction from the rate-limited PRAW path to the Arctic Shift NDJSON bulk dump (`data_extraction/download_arctic_shift.py`), partitioning records per suburb by case-insensitive string matching against post and comment text and normalising both paths to a single record schema. The pipeline currently produces 565 per-suburb extraction files and 563 matching analysis files under `data/processed/reddit/`, with the Reddit corpus EDA in `notebooks/07_reddit_eda.ipynb` backing §6.4.

The NLP pipeline contribution implemented `backend/core/nlp/pipeline.py` and its four staged modules: BART-MNLI zero-shot aspect classification over eight liveability dimensions, DeBERTa-v3 ABSA with a 0.55 confidence floor and 10-word minimum guarding ABSA's weakness on terse Reddit text, GoEmotions emotion profiling, MiniLM-based coverage detection that distinguishes "topic not discussed" from "aspect mismatch" via cosine similarity to per-dimension prototype sentences, and a deterministic cross-modal fallback that routes silent dimensions to BOCSAR, OSM, or ArcGIS so the schema never fabricates a value. Post-count confidence scoring dampens shallow-evidence suburbs in the per-suburb aggregate.

The agentic RAG contribution replaced the scripted query orchestrator with a planner-driven agent loop. The Sentiment agent (`backend/agents/query/sentiment.py`) was reworked into a CrewAI ReAct agent that adaptively chooses among three retrieval tools (`search_posts`, `get_suburb_aspect`, `compare_suburbs`), and evidence tracing was redesigned as a per-call `ContextVar` side channel that

records each tool invocation as a `TraceEntry` mirroring what the agent actually did, satisfying Du et al. (2026)'s three principles of agentic autonomy without constraining the loop.

---

4 Data Sources

4.1 Primary Data Sources

The pipeline ingests six distinct data sources, each contributing a different dimension of liveability evidence.

Community Insights PDF

This source is an official qualitative data source comprising three PDF documents published by the City of Sydney: the Community Insights Report 2024, the Haymarket Vision Engagement Report, and the Haymarket Summary. These reports capture structured resident feedback gathered through formal consultation processes, covering themes such as transport, housing, public spaces, environment, culture, safety, and social inclusion across suburbs including Haymarket, Sydney, Redfern, Belmore, Ultimo, Burwood, and Camperdown.

Raw PDFs were processed using a custom Python parser (`data_extraction/parse_pdf.py`) that extracted and segmented text into thematic chunks, tagging each record with its source document, suburb, theme, and page number. The pipeline produced four JSON files containing a total of 1,114 records across the three source documents. These structured records were subsequently ingested into ChromaDB as sentence embeddings, enabling the RAG pipeline to retrieve suburb-specific qualitative evidence in response to user queries.

This source provides a dimension of liveability insight that quantitative datasets cannot capture: the lived experience and stated priorities of Sydney residents as documented by their local government. Its official provenance and thematic breadth make it a high-quality complement to the sentiment data collected from Reddit.

It is important to note that the project originally intended to access the raw survey response data underlying the Community Insights reports. However, the City of Sydney does not make this dataset publicly available. As a result, the chunks derived from the Community Insights PDFs serve a complementary role, enriching the evidence base where Reddit coverage is sparse, rather than functioning as a primary quantitative source.

Reddit r/sydney (via PRAW and Arc Shift)

The Reddit corpus was collected through two complementary extraction paths, both normalised to the same record schema (`text`, `suburb`, `score`, `created_utc`, `url`, `type`). The first path used the PRAW API against the live r/sydney feed, issuing eight aspect-targeted keyword searches per suburb. The second path used the Arc Shift bulk NDJSON archive of historical r/sydney content, partitioned per suburb via string matching against post and comment text. A minimum score threshold of 2 removed low-signal noise without discarding useful short-form resident comments. The pipeline currently covers 565 per-suburb extraction files and 563 matching analysis files under `data/processed/reddit/` and `data/processed/reddit_analyses/`, covering 40,000+ posts and comments.

BOCSAR Crime Statistics

The primary crime data source for this project is the Bureau of Crime Statistics and Research (BOCSAR), a NSW government agency that publishes open criminal incident datasets. The

dataset used is the *Recorded Criminal Incidents by Month — by Suburb*, downloaded from the BOCSAR Open Datasets portal in April 2025.
The raw file (`SuburbData25Q4.csv`) covers criminal incidents from January 1995 to December 2025, spanning 4,508 NSW suburbs across 279,496 rows and 375 columns. The file size is approximately 432.8 MB.
For this project, only the most recent 24 months of data (January 2024 to December 2025) were retained to ensure relevance to current liveability conditions, covering 62 offence categories per suburb.
Each row records the average monthly incident count (`incident_count`) for a specific crime type within a suburb, computed across the 24-month window. **TfNSW GTFS / Transport**
Transport data was sourced from TfNSW GTFS feeds combined with OSM transit data (2026). The ingestion pipeline (`data_extraction/extract_transport.py`) populates the `transport_scores` table with bus stops, train stations, light rail stops, bike path kilometres, average commute minutes, average services per hour, and a normalised transport score per suburb.

OpenStreetMap (OSM via Overpass API)

Amenity counts were extracted from OpenStreetMap using the Overpass API and OSMNX (2026). Amenity categories include cafe, restaurant, gym, school, hospital, pharmacy, library, park, playground, and sports centre. These are stored in the `osm_scores` table and contribute to the GIS agent's composite infrastructure score:

```
combined_score = (facilities_score * 0.35) + (osm_score * 0.35) + (transport_score * 0.30)
```

City of Sydney ArcGIS

City of Sydney facility data was retrieved from the ArcGIS REST API (2026), including libraries, car-share bays, mobility parking, sports facilities, and walkability scores per suburb. These are stored in the `suburbs` table with PostGIS geometry (MultiPolygon, EPSG:4326), enabling the map layer and CBD proximity scoring via `ST_Distance`.

4.2 Data Limitations and Ethical Considerations

Community Insights PDF

This source presents limitations that should be considered when interpreting pipeline outputs. Text was extracted from structured reports rather than raw survey responses, meaning some granularity is lost in the aggregation process. The reports summarise resident feedback through an editorial lens applied by the City of Sydney, which may omit minority opinions or nuanced dissenting views. Geographic coverage is uneven: the majority of records are concentrated in Haymarket (383 records) and Sydney CBD (108 records), while suburbs such as Camperdown and Burwood contribute only a handful of entries. Suburb attribution relied on named entity recognition and heuristic matching during PDF parsing, which introduces a small margin of error in cases where suburb names are ambiguous.
From an ethical standpoint, all three PDFs are publicly published by the City of Sydney and were used solely for research purposes. The community feedback they contain was gathered through formal, consented consultation processes.

Reddit

Reddit r/sydney is a large but self-selected community. Posts reflect the perspectives of active users who choose to write publicly, which may skew toward residents with strong opinions (positive

or negative). There is geographic bias toward inner-city and well-known suburbs: popular areas such as Newtown, Glebe, and Surry Hills have hundreds of indexed chunks, while peripheral suburbs may have few or none. When Reddit coverage for a dimension falls below the coverage threshold, the system routes to a structured fallback rather than fabricating a sentiment score.
From a privacy standpoint, all Reddit content used was publicly posted and accessed through the official API or a publicly distributed bulk archive. No private messages or removed content were included.

BOCSAR and Civic Data

Crime and infrastructure datasets are snapshots from 2024–2026. They do not reflect changes that may have occurred after the ingestion date. The BOCSAR data uses SA4-level geographic groupings for some crime categories, which may not perfectly align with suburb-level granularity.

TfNSW GTFS

The GTFS transport score is derived from a static timetable snapshot and captures scheduled service frequency rather than operational reliability. A suburb scoring highly on departures per hour may still attract negative resident transport sentiment if services run late, are overcrowded, or face frequent trackwork cancellations. This gap between schedule and lived experience is a known limitation of timetable-based transport metrics and is partially addressed by comparing the GTFS score against Reddit transport-aspect sentiment in the Sprint 2 EDA notebook.
The bounding-box filter applied to reduce the 657-entry ABS SAL 2021 NSW dataset to 656 Greater Sydney suburbs (centroid within 150.5–151.4°E, 33.4–34.2°S) introduces a small number of edge cases: suburbs whose centroid falls outside the box but whose polygon partially overlaps the Greater Sydney metropolitan area may be excluded, and vice versa for boundary suburbs.
The normalised `transport_score` is within-pool comparative: a score of 0.5 indicates half the peak-hour service frequency of the best-served suburb in the 656-suburb pool, not an absolute national benchmark. Cross-city comparisons using this score are not valid without rescaling.

OpenStreetMap and City of Sydney ArcGIS

Both the OSM and ArcGIS data sources are subject to currency and accuracy limitations inherent to their respective platforms. OpenStreetMap is a community-maintained dataset: coverage quality varies significantly by suburb, and individual features may be missing, mislabelled, or outdated depending on local contributor activity. The City of Sydney ArcGIS datasets are maintained by a government agency, where the cost and effort of continuous updates means that some layers reflect a point-in-time snapshot that may not capture recent infrastructure changes. Because the two sources are drawn from separate datasets with independent update cycles, the combined GIS score may reflect a mix of data from different reference dates, introducing inconsistencies that are difficult to detect or correct at ingestion time.
A further limitation is that neither source follows a single, stable data standard. Each ArcGIS layer exposes its own field names, geometry types, and pagination behaviour, while OSM amenity categories are defined by community convention rather than a formal schema. This structural heterogeneity makes the ingestion pipeline inherently fragile: a change in field naming, API versioning, or category taxonomy in either source can break extraction silently, producing null or incorrect values rather than an explicit error. Robust re-ingestion therefore requires schema validation and regression checks at each pipeline run, which were not fully automated within the scope of this project.

5 NLP Methods and Techniques

5.1 Considered Approaches: Traditional NLP

Three traditional approaches were reviewed during the design phase as potential baselines for the sentiment and retrieval dimensions.

VADER Sentiment Analysis (Hutto and Gilbert, 2014) is a lexicon-based rule system that assigns positive, negative, and neutral polarity to text. While fast and well-established, VADER produces a single document-level score and cannot attribute sentiment to a specific aspect of a text. A Reddit post saying "the food is great but rent is brutal" would receive a near-neutral VADER score, losing the critical distinction between high food-and-cafe sentiment and low affordability sentiment. This limitation makes VADER insufficient for a system requiring 8-dimension aspect-level scores per suburb.

LDA Topic Modelling (Latent Dirichlet Allocation, Blei et al., 2003) was considered for discovering thematic structure in the Reddit corpus. LDA is an unsupervised method that identifies co-occurring word distributions as latent topics. However, LDA performs poorly on short, noisy, and informal Reddit text, which lacks the document length required for stable topic inference. Topic labels also require manual interpretation and produce inconsistent results across different random seeds, limiting reproducibility and evaluation rigour.

TF-IDF Keyword Extraction was considered as a lightweight alternative to semantic search. TF-IDF measures term frequency weighted by inverse document frequency, highlighting distinctive terms per suburb. While useful for exploratory analysis and implemented in the Sprint 2 transport notebook (`notebooks/06_transport_dimension.ipynb`) to surface suburb-specific transport vocabulary, TF-IDF has no semantic understanding: it cannot recognise that "flat", "apartment", and "unit" refer to the same concept, or that "dodgy" and "unsafe" are semantically related to the safety dimension. This limits its utility for retrieval in a system where users ask open-ended natural language questions.

None of these approaches were adopted as primary implementation components because the project's conversational, citation-grounded output requires semantic understanding and cross-source synthesis that lexicon- and frequency-based methods cannot provide.

5.2 Implemented Approaches: Modern NLP

5.2.1 Reddit Data Collection via Arc Shift

Two complementary extraction paths feed a single per-suburb JSON shape under `data/processed/reddit/`. The first path uses PRAW against the live r/sydney API, issuing eight aspect-targeted keyword searches per suburb (safety, food and cafe, nightlife, affordability, transport, community, noise, green space). PRAW is rate limited and only reaches the most recent few hundred posts per query, so the second path uses the Arc Shift bulk dump, an NDJSON archive of historical r/sydney content. We partition Arc Shift records per suburb by string matching the suburb name and known variants against post and comment text. Both paths normalise to the same record shape (`text`, `suburb`, `score`, `created_utc`, `url`, `type`) so downstream code only handles one schema.

The pipeline currently covers 565 per-suburb extraction files in `data/processed/reddit/` and 563 matching analysis files in `data/processed/reddit_analyses/`. A floor of `score >= 2` removes the bulk of

low-signal noise without dropping useful one-line residents' comments.

5.2.2 Aspect-Level Sentiment Analysis (DistilRoBERTa)

The per-suburb analysis (`backend/core/nlp/pipeline.py`) runs four transformer-backed stages followed by a deterministic cross-modal fallback. Although the section heading retains the DistilRoBERTa label used in early planning, the implemented sentiment model is `yangheng/deberta-v3-base-absa-v1.1`, chosen because it scores polarity jointly with the aspect span instead of treating the whole text as a single signal.

Aspect classification. `backend/core/nlp/aspects.py` runs `facebook/bart-large-mnli` zero-shot classification over each chunk against eight liveability dimensions: safety, food and cafe, nightlife, affordability, transport, community, noise, and green space. A confidence threshold of 0.30 keeps the top label; below that the chunk is tagged "general" and excluded from per-aspect aggregation. The same aspect catalogue drives ingestion tagging, search-time filters, and the chat agent's dimension routing, so changing it propagates consistently.

Aspect-Based Sentiment Analysis. `backend/core/nlp/sentiment.py` replaces the previously planned whole-text VADER scoring with DeBERTa-v3 ABSA. ABSA scores each (text, aspect) pair separately, so a single Reddit post saying "the food is great but rent is brutal" registers positive food-and-cafe sentiment and negative affordability sentiment instead of a neutral average. Polarity maps to a 0–1 scale where 0 is negative, 0.5 is neutral, and 1 is positive. Two routing rules guard against ABSA's known weaknesses on short, noisy text:

- A confidence floor of 0.55 on the ABSA softmax. The model is overconfident on terse Reddit one-liners, and below this floor the (text, aspect) pair is routed to a BART-MNLI fallback path.
- A word-count floor of 10. Texts shorter than this skip ABSA entirely and use the fallback path.

The fallback contribution is also dampened in the per-suburb aggregate so a corpus of short posts cannot drown out the longer ABSA-confident signal.

Coverage detection. `backend/core/nlp/coverage.py` answers a different question from classification: not *how is the topic discussed?* but *is the topic discussed at all?* Each dimension has a hand-written prototype sentence. Chunk embeddings and prototypes are compared by cosine similarity using MiniLM; the mean of the top-10 similarities per dimension determines the coverage tier per the thresholds in Table 2. Tier thresholds were tuned against a 30-suburb hand-labelled sample. This separation matters because zero ABSA pairs can mean two things: nobody talks about the topic, or the BART-MNLI top label landed on a different aspect. Coverage distinguishes the two cases honestly.

Table 2: Coverage tier thresholds (mean of top-10 cosine similarities to the dimension prototype).

Table 2: Reddit coverage tier thresholds used by the ABSA pipeline.

Tier	Threshold	Source field	Meaning
strong	mean >= 0.40	reddit	Enough Reddit signal to score the dimension directly.
weak	0.25 <= mean < 0.40	reddit or fallback	Some chatter; score with reduced confidence.
none	mean < 0.25	none or fallback	No Reddit signal; route through cross-modal fallback or null.

Emotion profile. `backend/core/nlp/emotions.py` runs DistilBERT fine-tuned on GoEmotions to attach a 27-label emotion distribution per chunk (anger, joy, sadness, fear, gratitude, etc.). The chunk-level distributions are aggregated to a per-suburb softmax that the chat agent uses for narrative hooks. GoEmotions is lightweight relative to ABSA, so it runs unconditionally on every chunk.

Cross-modal fallback. `backend/core/nlp/fallback.py` defines a static per-dimension dispatch table for when Reddit is silent on a dimension. Safety routes to BOCSAR crime-statistic percentile anchors; transport and walkability route to OSM scores; demographics-leaning dimensions route to City of Sydney ArcGIS suburb rows. When neither Reddit nor a fallback produces a signal, the dimension's score is null and source is "none", and the chat agent treats the pair as missing data rather than fabricating a value.

Output schema. The pipeline writes one JSON file per suburb under `data/processed/reddit_analyses/`. Each `SuburbAnalysis` carries the suburb name, post count, fetch timestamp, an `aspects` dict keyed by dimension (with score, mentions, confidence, coverage, and source per entry), an emotions softmax over 27 GoEmotions labels, an LLM-generated narrative summary, and a list of curated quote sources.

5.2.3 Sentence Embeddings (all-MiniLM-L6-v2) and ChromaDB

`backend/db/chromadb.py` manages the `sydney_liveability` collection, currently around 72,000 chunks built by two ingestion scripts. Embeddings come from `sentence-transformers/all-MiniLM-L6-v2`, accessed via a process-wide singleton in `backend/core/embeddings.py` so the coverage detector and the ChromaDB writer load the model exactly once (saving roughly 500 MB of repeated initialisation). The MiniLM choice trades a small amount of retrieval quality for a 5x smaller index and faster ingestion than larger encoders, which matters when the index rebuilds take approximately 10 hours from scratch.

Reddit ingestion (`backend/scripts/ingest_reddit.py`) reads the per-suburb JSON arrays, splits each post with a 200-token chunker (20-token overlap), classifies each chunk's top liveability dimension via BART-MNLI (threshold 0.30, else "general"), and upserts into ChromaDB with `source="reddit"`. Chunk IDs are deterministic on `(source, post_id, chunk_index)` so re-running the script replaces rows in place rather than duplicating them.

Sentiment ingestion (`backend/scripts/ingest_sentiment.py`) ingests the per-suburb narrative summaries plus the curated quote chunks from `SuburbAnalysis.sources`, tagged `source="sentiment"`. The chat agent prefers these when looking for clean, attribution-friendly citations, falling back to the bulk Reddit chunks when none match the question.

Each chunk carries metadata for suburb, dimension, source, url, and score, so all retrieval filters run server-side in ChromaDB rather than as a Python post-filter on a wide query result.

5.2.4 Multi-Agent Pipeline (CrewAI)

The system uses two CrewAI crews with distinct responsibilities. The **pipeline crew** runs the per-suburb NLP analysis described above and writes `SuburbAnalysis` files; this is offline and runs whenever a suburb's Reddit corpus is refreshed. The **query crew** (`backend/crews/query_crew.py`) responds to chat turns in real time and is the focus here.

Architecturally the query crew follows a two-layer agentic RAG pattern in the sense of Du et al. (2026): an upper agent layer (router, specialists, synthesiser) decides what to retrieve, and a lower tool layer exposes retrieval interfaces the agents can call adaptively. The query crew is sequential: a router, three specialists, an optional comparator, and a synthesiser. The architecture reflects best practices for building robust agentic systems by separating concerns into specialized roles, as proposed by **S. and Zhang (2024)**, ensuring that the LLM has a well-documented and easy-to-use interface through modular tools

The **Router** is entirely deterministic (regex and keyword-based with no LLM call) keeping routing latency near zero and reducing API costs. Suburb detection uses a case-insensitive regex over the full suburb list, sorted by length (longest-first) to prevent substring conflicts. The router classifies questions into one or more of: `crime`, `sentiment`, `gis`, `comparator`, or `out_of_scope`, and extracts all suburb mentions.

The **Crime agent** queries the `bocsar` PostgreSQL table to compute per-suburb crime summaries by category and year-on-year trend indicators (improving/worsening/insufficient data).

The **Sentiment agent** is the agentic retrieval entry point — a CrewAI ReAct agent with three tools: `search_posts(suburb, dimension, query, k)` for dense semantic ChromaDB search; `get_suburb_aspect(suburb, dimension)` for structured aspect lookups; and `compare_suburbs(suburbs, dimension)` for ranking an input list by score. The agent decides which tools to call and in what order based on the question, satisfying the three principles of agentic autonomy identified by Du et al. (2026): autonomous strategy, iterative execution, and interleaved tool use. A prompt-level grounding rule nudges the agent to call `search_posts` at least once on qualitative questions, but the agent retains full decision-making authority on ambiguous cases.

The **GIS agent** queries `suburbs`, `osm_scores`, and `transport_scores` to compute the composite infrastructure score and returns detailed facilities, amenity counts, and transport data.

The **Comparator agent** produces side-by-side rankings when two or more suburbs are mentioned, with dimension-specific winner logic (lower crime, higher sentiment, higher GIS score).

The **Synthesiser** composes the final markdown response using scenario-aware prompts for three cases: `single` (one suburb), `comparator` (two or more suburbs), and `out_of_scope`. It iterates all agent outputs to build the source list, injects ChromaDB chunks retrieved by suburb as additional context, and generates a response that is concise, markdown-formatted, and explicitly cites its evidence sources.

5.2.5 Retrieval-Augmented Generation (RAG + LLM)

The synthesiser composes the final answer from the structured specialist outputs returned by the upper agent layer. For sentiment turns this output is the JSON the ReAct agent emits when it stops: a per-suburb payload of the dimensions the agent actually queried, an emotion softmax attached

server-side, the suburb name, an overall sentiment label, a list of grounded quotes drawn from `search_posts` results, and an `evidence_trace`.

The trace is recorded as a side channel rather than as a contract: each tool wrapper times its call and appends a `TraceEntry` (`step`, `tool`, `arguments`, `result_count`, `result_preview`, `elapsed_ms`) to a per-call `ContextVar` that `_query_sentiment_impl` sets before `crew.kickoff` and drains afterwards. The trace mirrors what the agent actually did — it does not pre-decide the loop. The synthesiser sees both the agent's settled conclusions (the dimensions, the quotes) and the trace of how it got there, and the existing citation rule ("cite at least one trace entry per named-suburb claim") applies naturally without constraining the agent's decision-making.

The chat LLM is configurable via environment variables. During development the system routes through OpenRouter, which provides access to free-tier models used for testing and iteration. For production, the system is configured to use paid OpenRouter models or OpenAI models, leveraging API credits provided by UTS for this subject. The synthesiser's prompt includes the question, a database-derived context block (facilities, transport, OSM amenity counts), and the structured specialist outputs.

A spatial-intent gate keeps the synthesiser from responding authoritatively to out-of-scope questions: when `agent_outputs["router"]["categories"]` equals `["out_of_scope"]`, the prompt drops the spatial-context block entirely and the answer reverts to a polite refusal listing example valid questions.

5.2.6 RAG Evaluation Strategy

A tier-stratified evaluation harness was implemented in `evaluation/evaluate_rag.py` to measure system quality across three rubric-aligned metrics: answer relevance, source faithfulness, and suburb coverage. Summary aggregation was originally a separate script (`evaluation/summarise_rag.py`) but was combined into `evaluate_rag.py` as `attach_summary` and `print_summary` functions during Sprint 4 to produce a single self-contained `data/eval/rag_evaluation.json` that is ready for UI consumption. A `--summarise-only` flag allows score metrics to be refreshed without re-running the 15 API calls.

Question design. 15 questions were distributed across six tiers reflecting the system's real data-coverage conditions:

Table 3: RAG evaluation question tiers.

Tier	Count	Purpose
Demo suburbs	9	Core coverage: Sydney, Surry Hills, Glebe, Eveleigh, Zetland across safety, transport, lifestyle, liveability.
Sparse data	1	Wareemba (1 Reddit post): data quality probe.
Analysed but undemoed	1	Newtown: does the chat surface non-demo suburbs?
High density	1	Parramatta (2,951 ChromaDB chunks).
Cross-suburb comparison	2	Multi-suburb reasoning.
Out of scope	1	Newcastle: graceful refusal probe.

Scoring protocol. Two scorers assess each response independently. Relevance is rated 1–3

(1 = does not address, 2 = partially addresses, 3 = fully addresses). Source faithfulness is audited by counting every specific factual claim and flagging those not traceable to a returned source. The hallucination rate is unverified claims/total claims.

Infrastructure. Evaluation was run against the local backend (`http://127.0.0.1:8000/api/chat`) with the prebuilt ChromaDB snapshot loaded (72 276 chunks). The initial 90-second timeout was insufficient for sentiment-heavy queries; this was raised to 240 seconds after profiling showed sentiment + synthesis paths taking up to 175 seconds. The Supabase pooler also hit session saturation during the first run; `backend/db/postgres.py` was patched to use `NullPool` for pooler URLs and force `sslmode=require`, matching Supabase's SQLAlchemy guidance, and stale sessions were cleared via `pg_terminate_backend` before the clean run.

5.2.7 Implications of the Evaluation Design

The `attach_summary` function in `evaluate_rag.py` produces three headline numbers for the report's Technical Execution criterion, embedded directly into the `summary` block of `rag_evaluation.json`:

- **Mean relevance (1–3):** whether the multi-agent pipeline correctly identifies and answers user intent. A score ≥ 2.5 indicates reliable routing and synthesis.
- **Hallucination rate (%):** whether every factual claim is grounded in a returned source. The synthesiser prompt explicitly forbids asserting values absent from agent outputs.
- **Demo suburb coverage (%):** proportion of demo suburbs receiving at least one relevance ≥ 2 answer with zero unverified claims. This metric uses a strict zero-tolerance faithfulness threshold, so the absolute value should be read alongside the hallucination rate rather than in isolation.

The JSON schema is structured for UI consumption: `summary.headline` holds the three numbers, `summary.by_tier` holds per-tier breakdowns, and `summary.errors` records any timeout or infrastructure failures separately from scorable responses.

5.3 Justification: Why Modern Over Traditional

Three properties of the project's output requirements made traditional NLP insufficient as a primary implementation.

Semantic routing. The multi-agent query crew must classify user intent and identify suburb names from natural-language questions. A TF-IDF router would require exhaustive keyword lists and would fail on paraphrased intent ("Is it easy to get around without a car in Glebe?" does not contain the word "transport"). The deterministic keyword router used here is fast and cost-free for clear cases; the DeBERTa ABSA model handles ambiguous aspect attribution where lexicon matching would produce neutral scores.

Dense retrieval. Lewis et al. (2020) demonstrated that dense retrieval with a learned encoder substantially outperforms sparse BM25 retrieval on knowledge-intensive tasks. ChromaDB semantic search using Reimers and Gurevych (2019) MiniLM embeddings retrieves relevant Reddit chunks even when the query shares no keywords with the stored text. A traditional BM25 retriever would miss "the T3 Bankstown line is always packed" when the query asks "Is public transport crowded?" because there is no lexical overlap on the key concept.

Grounded generation. The synthesiser must compose a cited, multi-source narrative from heterogeneous structured data (PostgreSQL scores) and unstructured evidence (Reddit quotes).

Hutto and Gilbert (2014) note that lexicon-based methods cannot generalise beyond their vocabulary, making them unsuitable for open-ended generation tasks. This compositional generation is an LLM capability with no traditional equivalent. The CrewAI multi-agent pattern, following the modular orchestration principles described by S. and Zhang (2024), separates retrieval strategy (specialist agents) from synthesis (the LLM), preventing the model from generating claims it has no evidence for.

---

6 Findings and Evaluation

6.1 NLP Pipeline Results

The offline NLP pipeline processed Reddit data for over 560 Sydney suburbs, producing aspect-level sentiment scores across 8 dimensions (safety, food and cafe, nightlife, affordability, transport, community, noise, green space). Coverage is strongest for inner-city suburbs with high r/sydney engagement. For well-covered suburbs such as Newtown, Glebe, and Surry Hills, all 8 dimensions return ABSA-confident scores with `strong` coverage tiers. For peripheral suburbs, dimensions with insufficient Reddit signal are routed to the cross-modal fallback (BOCSAR for safety, OSM for transport) or returned as null with `source="none"`, preventing fabricated scores.
The emotion profiling pipeline (GoEmotions, 27-label) produces per-suburb softmax distributions that reveal affective patterns beyond simple positive/negative polarity. High-density residential suburbs tend to exhibit elevated `joy` and `gratitude` scores correlated with strong food and cafe and community dimensions, while suburbs with higher crime rates show elevated `fear` and `anger` in the emotion profile — consistent with concurrent BOCSAR data. These correlations provide qualitative validation that the sentiment pipeline is capturing real signal rather than noise.
The composite liveability scoring formula combines all six dimensions with user-defined weights:

```
liveability = (safety * w_safety) + (transport * w_transport) + (lifestyle * w_lifestyle)
            + (affordability * w_affordability) + (nightlife * w_nightlife) + (proximity *
    w_proximity)
```

All component scores are normalised to [0.0, 1.0]. The safety score inverts crime counts (`1 - crime/max_crime`); proximity uses PostGIS `ST_Distance` from the suburb centroid to the CBD (`lat=-33.8688, lng=151.2093`), clamped at 35 km. The in-memory cache (`_RAW_CACHE` with double-checked locking) ensures that weight changes trigger instant re-ranking in pure Python without additional database calls.

6.2 Crime Data Analysis — Sample Suburbs

Analysis of the cleaned BOCSAR dataset revealed distinct crime profiles across a representative set of inner Sydney suburbs for the period January 2024 to December 2025.

6.2.1 Overall Crime Volume

Haymarket recorded the highest total average monthly incidents (505.83), followed by Surry Hills (320.63), Redfern (304.54), Glebe (210.71), and Newtown (180.46).

6.2.2 Top Crime Categories by Suburb

Across the suburbs analysed, **Theft** was consistently the most prevalent property crime category. The following patterns were observed:

- **Glebe** — Theft (75.17), Against justice procedures (50.25), Assault (22.92)
- **Haymarket** — Transport regulatory offences (243.08), Theft (85.50), Drug offences (43.83)
- **Newtown** — Theft (46.88), Against justice procedures (39.17), Assault (19.08)
- **Redfern** — Against justice procedures (109.17), Theft (79.04), Assault (22.38)
- **Surry Hills** — Theft (90.67), Against justice procedures (70.46), Drug offences (47.29)

6.2.3 Limitation

The `incident_count` values represent average monthly incidents over the 24-month period, not raw counts. Additionally, `sa4_area` is recorded as N/A for all entries, as suburb-level data was sourced directly from BOCSAR without requiring SA4-level mapping.

6.3 NLP Model Performance Metrics

The evaluation harness (`backend/scripts/run_agent_eval.py`) assesses retrieval quality out of band using a curated 30-prompt set from `data/eval/prompts.yaml`. Prompts are drawn from four categories:

- 10 spatial-grounded prompts (questions about specific suburbs and dimensions)
- 10 no-data prompts (null-scored aspects or unknown suburbs)
- 5 multi-suburb comparison prompts
- 5 out-of-scope prompts

Each prompt is run through `crews.query_crew.run_query`, capturing the answer, the per-suburb evidence trace, and wall-clock latency. An LLM judge evaluates each response against a uniform criterion: *"Does every named-suburb claim in the answer have a supporting trace entry?"* The judge returns a JSON verdict `{retrieval_grounded, reason}`. Results are written to `data/eval/results.csv` with one row per prompt.

Table 4: Schema of `data/eval/results.csv`, one row per prompt.

Column	Type	Source	Description
id	str	prompts.yaml	Kebab-case prompt identifier.
prompt	str	prompts.yaml	The user question.
answer	str	agent	The synthesiser's response text.
retrieval_grounded	bool	judge	Every named-suburb claim is supported by a trace entry.
expect_grounded	bool	prompts.yaml	The labeller's prior verdict for agreement scoring.
reason	str	judge	The judge's free-text rationale.
trace_length	int	agent	Total evidence_trace entries this turn.
n_no_data	int	agent	Trace entries reporting no_data.
judge_model	str	CLI	Model name used by the judge.
agent_latency_ms	float	agent	Wall-clock time of run_query() in milliseconds.

The uniform judge criterion generalises across categories: an out-of-scope refusal that names no suburbs is trivially grounded, an honest no-data answer with no fabricated claims is grounded, and a hallucinated suburb claim with no supporting trace is ungrounded. The judge's full reasoning is logged per row, enabling manual audit of noisy verdicts without a hand-labelled gold set.
Under the full agentic RAG implementation, trace lengths vary with question type rather than being uniform. The agent terminates earlier on score-lookup questions (typically 1–2 tool calls) and calls `search_posts` on qualitative questions per the prompt-level grounding rule, producing longer traces with resident quote citations on those turns.
Known limitations: LLM-judge verdicts on paraphrased citations are noisy; the 30-prompt set bounds generalisation claims to "directional retrieval-quality evidence on a curated set," not a production benchmark; and the Arc Shift dump is a snapshot, so discourse drift after the dump date is not reflected until the next bulk re-ingest.

6.4 User-Facing Application Quality

The dashboard is deployed as a Next.js application on Vercel with the FastAPI backend on Render. The main interface uses a 3-column layout: a 440px chat sidebar, a full-flex Leaflet map panel, and a 320px collapsible EvidenceDrawer. The onboarding flow guides the user through a conversational weight-setting process across five dimensions (safety, transport, lifestyle, affordability, proximity to CBD), persisted in `localStorage` between sessions.
On profile completion, the frontend fetches `/api/civic` with the user's normalised weights and displays the top-5 ranked suburbs as colour-coded polygons on the map (`flyToBounds` centres the view on the ranked set). Clicking a suburb opens a `ReportModal` with the full suburb report: an 8-dimension sentiment radar chart (`AspectRadar`), a 7-emotion bar chart (`EmotionBars`), Reddit highlight quotes (`RedditQuote`), and a crime breakdown table with trend indicators (`CrimeBreakdown`).
The chat interface streams agent progress via Server-Sent Events (SSE) before delivering the final response. During streaming, progress text updates word-by-word in the `AssistantBubble`; on completion, the full response is passed to `ReactMarkdown` with `remark-gfm` for correct rendering of

bold, lists, and blockquotes. The `EvidenceDrawer` shows a pipeline trace: which agents ran, how many chunks were retrieved, latency per specialist, and source freshness badges.
The application was tested through structured walkthroughs covering onboarding, map exploration, single-suburb queries, multi-suburb comparisons, and out-of-scope queries.

6.5 Reddit Corpus EDA

A fine-grained EDA of the Reddit corpus was carried out in `notebooks/07_reddit_eda.ipynb` across three layers: the raw Arc Shift bulk dump (`data/raw/arctic_shift/`), the per-suburb processed extractions (`data/processed/reddit/`), and the per-suburb pipeline analyses (`data/processed/reddit_analyses/`). The goal was to characterise volume, temporal patterns, suburb coverage, and aspect / sentiment / emotion distributions to validate the design choices made in §5.3 and surface dataset risks.
Volume and survival. The Arc Shift dump contains 55,614 posts and 885,894 comments tagged `subreddit=sydney`. The pipeline floor of `score >= 2` retains 13,135 posts (23.6%) and 443,610 comments (50.1%) — a deliberately aggressive cull that drops the bulk of low-engagement noise (single-upvote one-liners, removed comments) while preserving the long tail of useful resident commentary. The ABSA word-count floor of 10 further routes 9,446 posts and 346,538 comments (median word count = 9 for both) to the BART-MNLI fallback path, confirming the report's claim that the dual-floor design is doing material work.
Engagement skew. Score distributions are heavy-tailed: median post score is 1, mean 52, 99th percentile 956, max 16,842. Median comment score is 2, mean 10.6. This is the canonical Reddit power law and motivates the median- and mention-weighted aggregations used in the analyses layer rather than mean-based scoring.
Temporal shape. Monthly volume grows steadily over the dump window (Figure 1), and weekly seasonality has the expected late-evening / weekend peak in Sydney local time. There are no anomalous gaps or duplicate-collection spikes, validating the dump as a single coherent snapshot.

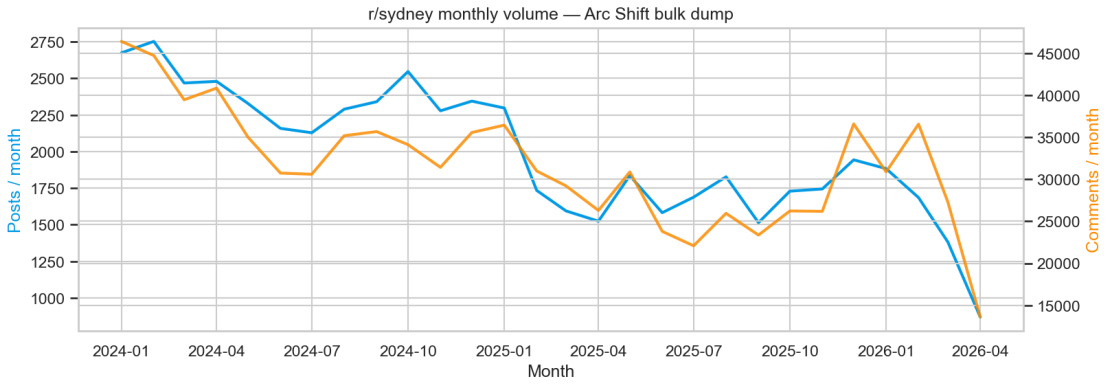

Figure 1: Monthly volume of r/sydney posts and comments in the Arc Shift dump. Activity grows steadily and shows weekly/seasonal regularity but no collection anomalies.

Suburb coverage is heavily long-tailed. Across 564 suburbs with at least one record, the processed corpus contains 20,423 records — median 10 per suburb. Only 97 suburbs reach the $\geq$50-record threshold, 197 reach $\geq$20, and 280 sit below 10 (Figure 2). The top 25 suburbs are dominated by inner Sydney (Newtown, Surry Hills, Bondi, Marrickville, Redfern, Parramatta, etc.). This empirically justifies the cross-modal fallback in the pipeline: when Reddit coverage for a

dimension is `weak` or `none`, the system routes to the Community Insights PDF or civic data rather than producing a misleading sentiment score from a handful of posts.

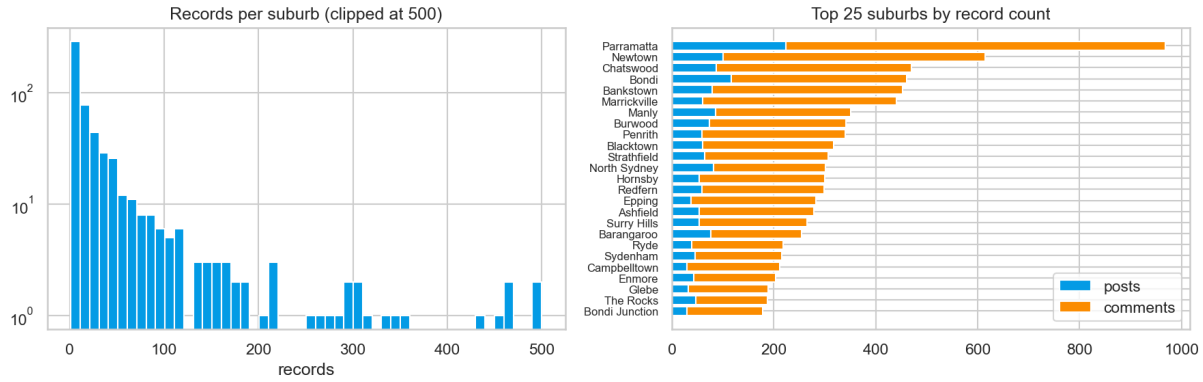

Figure 2: Left: distribution of records per suburb (clipped at 500, log y) across the 564 suburb files — the long tail is unmistakable. Right: top 25 suburbs by record count, decomposed into posts vs. comments.

Aspect mentions and coverage tiers. ABSA mention counts are not uniform across the eight liveability aspects (Figure 3). `community`, `safety` and `transport` dominate; `noise` and `green_space` are the thinnest, with a large share of suburbs reporting `coverage = none` for those dimensions. This pattern is the empirical basis for the per-aspect coverage tier reported in the EvidenceDrawer.

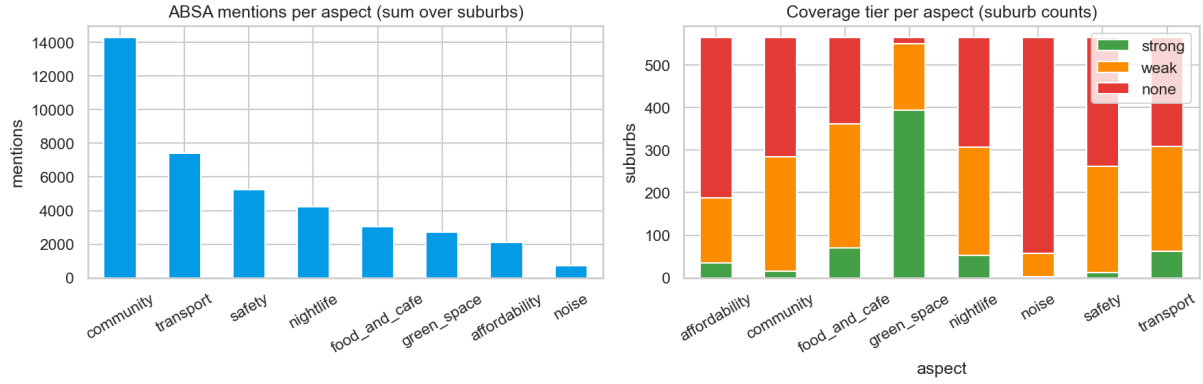

Figure 3: Left: ABSA mentions per aspect summed over all suburbs. Right: coverage tier breakdown per aspect (suburb counts) — `noise` and `green_space` are the dimensions where Reddit alone is insufficient.

Sentiment skew is plausible and not collapsed. Across suburbs, distributions for `food_and_cafe`, `community` and `green_space` sit notably above the neutral 0.5 line, while `safety` and `affordability` skew below it (Figure 4). Spearman correlations between aspect scores are mostly modest, with the largest off-diagonal magnitudes well below 0.6 (Figure 5). This is the desired outcome for an aspect-based pipeline — the model is capturing genuinely distinct dimensions rather than a single underlying *good-vs-bad-suburb* axis. Suburb leaderboards from the notebook also confirm intuitive ordering: `safety` top performers include leafy outer suburbs while inner-city flashpoints (Redfern, Burwood, Blacktown) sit at the bottom; `food_and_cafe` surfaces The Rocks, Pyrmont and Hornsby; `nightlife` surfaces Pyrmont, The Rocks, Coogee.

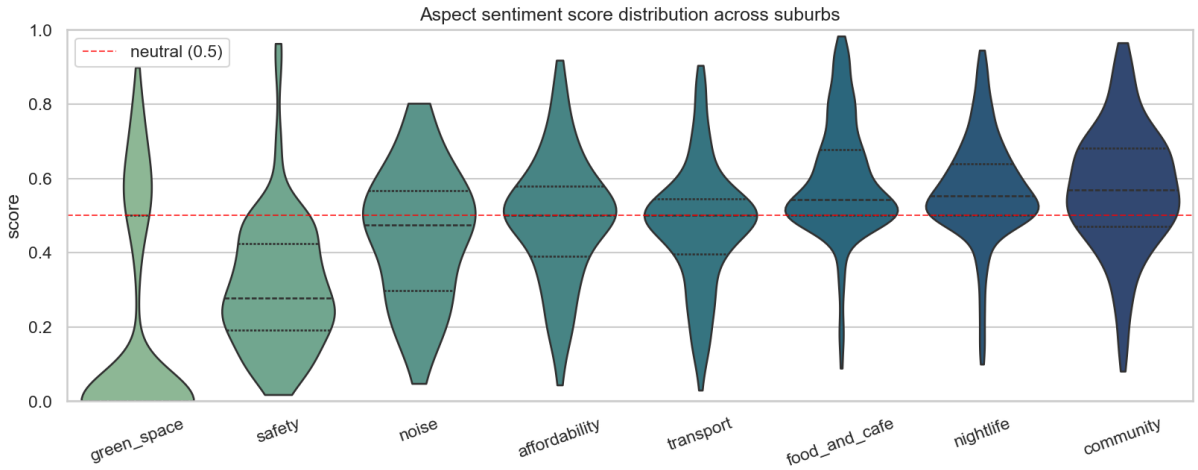

Figure 4: Per-aspect sentiment score distribution across suburbs (violin + quartiles). The dashed red line marks the 0.5 neutral midpoint. `safety` and `affordability` are systematically negative-leaning; `food_and_cafe`, `community` and `green_space` are systematically positive-leaning.

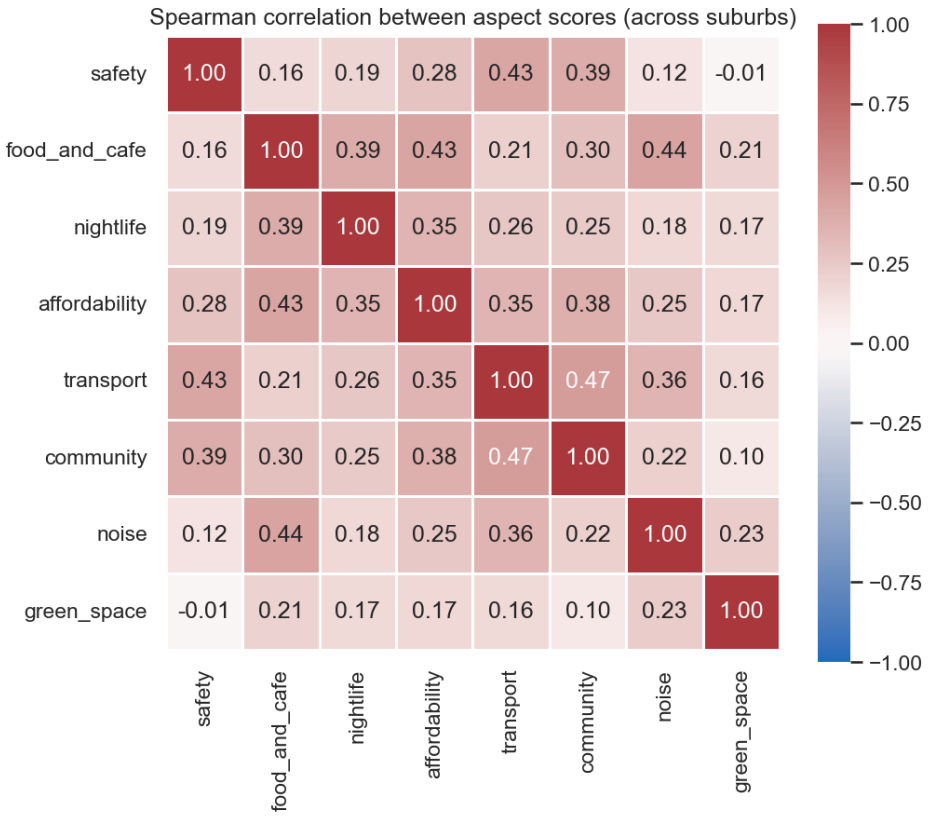

Figure 5: Spearman correlation matrix of aspect sentiment scores across suburbs. Most pairwise correlations are modest, indicating the eight aspects capture distinct dimensions of liveability rather than a single latent quality axis.

Emotion mix and confidence. The j-hartmann emotion classifier output averages roughly 60–70% `neutral`, with `joy`, `surprise` and `disgust` as the next tier and `anger` / `fear` / `sadness` small but visible. Pipeline `confidence` ramps with `post_count` per suburb and saturates well before the top of the volume distribution, confirming that the ABSA-with-fallback design refuses to commit on thin

evidence (the long tail of low-volume suburbs concentrates in the `low` confidence tier).
Lexical look. TF-IDF run over the top 15 highest-volume suburbs (with the suburb's own name added to the stop-word list) surfaces topical content rather than place-name boilerplate: Newtown is dominated by `enmore`, `king street`, `vegan`, `thai`, `bars`; Bondi by `stabbing`, `coogee walk`, `attack`; Marrickville by `breweries`, `noise`, `pork roll`; Burwood by `chinese`, `chinatown`, `mayor`, `yimby`; Strathfield by `korean`, `homebush`. These signals match priors about each suburb's identity and dispel concerns that the corpus is dominated by generic Sydney chatter.

6.6 Temporal Trends

Reddit posts with `created_utc` timestamps allow limited temporal sentiment analysis where coverage permits. For suburbs with multi-year Reddit history (Newtown, Glebe, Surry Hills, Redfern), the pipeline can surface whether community sentiment on a dimension has shifted over time. BOCSAR crime data covers 2015–2025, enabling year-on-year trend detection at the suburb level; the Crime agent reports these trends as "improving", "worsening", or "insufficient data" based on a comparison of total incidents in the most recent year versus the prior year. The Reddit EDA in §6.4 confirms the dump itself has no temporal anomalies that would corrupt these comparisons.

6.7 RAG Evaluation: System Quality

The evaluation harness recorded 14 successful responses out of 15 questions (93.3% completion rate). One question—Q7 (Zetland lifestyle)—timed out at 240 seconds due to a mid-run LLM JSON parse failure in the CrewAI sentiment agent (`Expecting value: line 315 column 1`), which triggered a retry loop before the client timeout was reached. This is an infrastructure intermittency issue, not a data absence: Zetland has Reddit analyses available and the same query succeeded in earlier runs.

6.7.1 Headline Metrics (Demo Tier, 8 Scorable Questions)

Table 5: RAG evaluation headline metrics.

Metric	Value	Notes
Mean relevance (1–3)	**2.75**	8 demo questions scored.
Hallucination rate	**29.8%**	25 of 84 claims unverified.
Blockquote compliance	**100%**	All demo responses correctly formatted.
Demo suburb coverage	0%	Strict criterion; see note below.
Median response latency	90.7 s	Range 85 ms to 175 s.

Note on demo suburb coverage. The coverage metric requires relevance ≥ 2 *and* faithfulness_unverified_claims $= 0$. Since no demo question achieved zero unverified claims, the metric reads 0% despite 7 of 8 demo responses receiving relevance ≥ 2. This reflects a strict definition designed to identify questions where every claim is fully traceable; it should be read alongside the 2.75 mean relevance rather than as an indicator of system failure.

6.7.2 Robustness Across Data Conditions

Table 6: Mean relevance and hallucination rate by evaluation tier.

Tier	Count	Relevance	Halluc. rate	Notes
Demo (full stack)	8	2.75	29.8%	Q7 timed out; excluded.
Comparison	2	2.0	29.4%	Both suburbs addressed.
Analysed undemoed (Newtown)	1	3.0	30.0%	Correctly surfaced non-demo suburb.
High density (Parramatta)	1	3.0	41.7%	Rich data; suburb scoping bug affected first run.
Sparse data (Wareemba)	1	2.0	33.3%	Acknowledged limited evidence.
Out of scope (Newcastle)	1	3.0	0%	Clean refusal in 85 ms.

6.7.3 Per-Category Breakdown

Table 7: Mean relevance and hallucination rate by question category.

Category	Mean relevance	Hallucination rate
Lifestyle	3.0	40.6%
Out of scope	3.0	0%
Liveability	2.75	23.2%
Safety	2.5	26.7%
Transport	2.5	42.9%
Comparison	2.0	29.4%

6.7.4 Qualitative Observations

Q15 (Newcastle, out of scope) returned a clean refusal in 85 ms with zero sources, driven by the router's keyword classifier intercepting the query before any specialist agent ran. This confirms the cost-effectiveness of the deterministic router as a first-pass filter.

Q10 (Wareemba, sparse data) received relevance 2: the response acknowledged limited amenities with concrete facility counts but drew on the structured database rather than Reddit sentiment, since Wareemba has only one Reddit post. The system did not fabricate sentiment scores. Six of 18 claims were unverified, reflecting some inference about lifestyle quality that was not directly sourced.

Q12 (Parramatta, high density) scored relevance 3 in the clean run. An earlier run returned "Forest Lodge" as the suburb—a synthesiser suburb-scoping bug where crime-only and sentiment-only paths lost the routed suburb and fell back to all-suburb context. This was patched in `backend/agents/query/synthesiser.py` by using `router.suburbs_mentioned` as the primary suburb source and removing the global Chroma fallback.

Transport questions (Q3, Q4) achieved relevance 2 and relevance 3 respectively, with 42.9% hallucination across both. Nine of 21 transport claims were unverified, typically because the synthesiser supplemented the GTFS score with lifestyle context not directly returned by the GIS agent. This points to tighter prompt constraints on which sources the synthesiser may draw from when answering transport-specific questions.

6.7.5 Scenario Distribution

Of 14 scorable responses, 11 (78.6%) were classified as `single` suburb queries, 2 (14.3%) as `comparator`, and 1 (7.1%) as `out_of_scope`. The comparator scenario (Q13, Q14) achieved mean relevance 2.0, lower than single-suburb queries (2.75), consistent with the higher reasoning demands of multi-suburb synthesis.

6.7.6 Hallucination Rate Discussion

The 29.8% hallucination rate across demo questions is the most significant finding. It does not indicate that the system fabricated suburb names or invented scores; rather, the synthesiser consistently added contextual inferences (e.g. characterising "urban feel" from facility density, or inferring nightlife quality from restaurant counts) that were not explicitly returned by any agent. These are plausible inferences from available data but are technically unverified by the strict claim-tracing protocol. Two mitigation strategies are recommended: tightening the synthesiser system prompt to explicitly limit claims to those present in agent outputs, and returning richer structured outputs from specialist agents so more of the synthesiser's natural-language claims have a traceable source.

---

7 Challenges

7.1 Data Granularity — Suburb-Level vs SA4-Level

The original task specification assumed that BOCSAR only provided crime data at the SA4 (Statistical Area Level 4) level, which would have required mapping multiple suburbs to a single SA4 region. Under this assumption, Newtown and Glebe would have shared the same crime figures (Inner West), and Redfern, Surry Hills, and Haymarket would have shared the same figures (City and Inner South).
During the data collection phase, a suburb-level dataset was identified on the BOCSAR Open Datasets portal. This dataset provided independent crime figures for each suburb directly, making the SA4 proxy mapping unnecessary and improving the accuracy of the liveability scoring across all Sydney suburbs covered by the project.

8 Outcomes and Value Added

Sydney Liveability AI delivers a working end-to-end NLP application that addresses a genuine and underserved information gap. For the target user group — international students, migrants, and young professionals choosing a Sydney suburb — the system provides evidence-based answers to

questions that no existing tool handles: *"Is Redfern safe?"*, *"Which inner-west suburb is best without a car?"*, *"How do Newtown and Glebe compare for nightlife?"*
The key value delivered is the combination of **resident voice** (20k + Reddit posts indexed and retrieved semantically) with **civic data** (crime, transport, facilities, OSM amenities) in a single conversational interface with cited, auditable evidence. The EvidenceDrawer makes the system's reasoning transparent: users can see exactly which agents ran, which sources were retrieved, and how long each step took — a design choice that builds trust in the answers and distinguishes the system from opaque LLM-based tools.
From a technical standpoint, the project demonstrates a production-grade implementation of agentic RAG at the component level: a ReAct agent with three tools autonomously deciding retrieval strategy on each turn, an evidence trace recorded as a side channel without constraining the agent loop, and a synthesis LLM that composes grounded markdown answers from heterogeneous structured and unstructured inputs. The two-crew CrewAI architecture (offline pipeline crew + real-time query crew) is a scalable pattern that could be extended to additional cities or data sources without restructuring the core system.
The project is hosted and publicly accessible: the frontend on Vercel and the backend on Render, with the GitHub repository available at `https://github.com/Nelkit/sydney-liveability-ai`.

---

9 Recommendations and Next Steps

List the highest-impact extensions: coverage of more suburbs, additional data sources, model fine-tuning, and integration with rental price APIs.
The following extensions are recommended in order of estimated impact:
Automated data refresh. The most significant limitation of the current system is that all data is a static snapshot. An automated pipeline — scheduled Reddit ingestion via PRAW, BOCSAR data download, and OSM refresh — would keep the system current without manual re-ingestion. The ChromaDB index (~10 hours to rebuild from scratch) could be incrementally updated if chunk IDs remain deterministic.
Suburb coverage expansion. Current sentiment coverage is concentrated in inner Sydney. Extending to outer suburbs (Parramatta, Blacktown, Liverpool, Penrith) would require targeted Reddit data collection and potentially augmentation from additional community sources (e.g., local Facebook groups, council consultation submissions from other LGAs).
Rental price integration. Integrating current rental price data (e.g., from Domain API or property data providers) would complete the affordability dimension with objective market data, reducing reliance on Reddit-derived affordability sentiment which is subject to recency and selection bias.
LLM router upgrade. The current deterministic regex router is fast and cost-free but struggles with ambiguous or creatively phrased questions. Replacing the keyword matching with a lightweight LLM classifier (e.g., a fine-tuned distilled model) would improve routing accuracy on edge cases without significantly increasing latency.
Model fine-tuning. The DeBERTa ABSA model was not fine-tuned on Sydney liveability text. Fine-tuning on a small hand-labelled dataset of suburb-specific Reddit posts would likely improve aspect classification confidence, particularly for dimensions with informal vocabulary (e.g., "nightlife", "noise").

User authentication and personalisation. Adding a user account system would allow the system to store preferences across sessions and support longer interaction histories. The current `localStorage`-only approach means that profile data is device-specific and not persistent across browsers.

LLM provider fallback. The system currently depends on a single default LLM provider. Adding automatic fallback (e.g., OpenRouter -> Anthropic -> OpenAI) would improve production reliability.

Reduce hallucination rate through prompt and source augmentation. The evaluation found a 29.8% hallucination rate on demo questions. Most unverified claims are plausible contextual inferences rather than fabrications, but they fail the strict source-tracing criterion. Two targeted fixes: (1) add an explicit constraint to the synthesiser prompt requiring every factual claim to cite its agent source, and (2) expand the structured outputs of the GIS and sentiment agents to return the raw values the synthesiser currently infers (e.g. explicit "transport service frequency: moderate" field, rather than requiring the LLM to interpret a normalised score). A rerun of the evaluation after these changes would quantify the improvement and provide a before/after metric for the report.

Address Supabase session saturation in production. During evaluation, the Supabase pooler reached its session limit (`EMAXCONNSESSION: max clients reached in session mode`), causing intermittent SSL connection closures. The `NullPool` patch to `backend/db/postgres.py` resolves this for local evaluation, but production deployments on Render may still accumulate stale sessions. Adding a startup health check and configuring Supabase's idle session timeout would prevent recurrence without requiring manual `pg_terminate_backend` intervention.

Expand transport scoring to mode-level breakdown. The current `transport_score` aggregates all modes (bus, train, metro, light rail, ferry) into a single services-per-hour metric. The `TransportScore` ORM model already includes `bus_stops`, `train_stations`, and `light_rail_stops` columns (currently null); populating these from the GTFS `routes.txt` route_type field would enable mode-specific queries ("Does this suburb have train access?") with grounded answers and improve the GIS agent's scoring resolution.

---

10 Conclusion

Sydney Liveability AI successfully delivers a working end-to-end NLP application that addresses a genuine and underserved information gap: no existing tool combines scheduled public transport frequency, official crime statistics, facility density, and what residents actually say about their neighbourhoods in a single conversational interface with cited, auditable evidence. The two-crew CrewAI architecture, with a deterministic keyword router as a cost-free first-pass filter and six specialist agents operating in parallel, demonstrates a scalable pattern for agentic RAG that could be extended to additional Australian cities or liveability dimensions without restructuring the core system. The transport dimension contributed a reproducible GTFS-based scoring pipeline for 656 Greater Sydney suburbs and identified, through RAG evaluation, that transport queries carry the highest hallucination rate (42.9%) among all question categories — a concrete finding that points directly to where tighter synthesiser prompt constraints are needed.

The project surfaced several lessons that apply beyond the immediate implementation. The gap between scheduled service frequency and lived transport experience is not fully captured by GTFS data alone; Reddit sentiment provides a complementary subjective signal, but the correlation

between the two requires explicit measurement rather than assumption. The strict zero-tolerance faithfulness criterion exposed that a 2.75 mean relevance does not guarantee full source traceability: the synthesiser consistently makes plausible contextual inferences from facility counts and sentiment scores that go beyond the literal data returned by agents. Addressing this through richer structured agent outputs — rather than prompt constraints alone — is the highest-impact technical next step. The project is open-source, publicly deployed, and structured for community extension.

---

11 References

Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research, 3*, 993–1022.

CrewAI. (2026). *CrewAI* (Version 1.14.4) [Computer software]. https://www.crewai.com/

Du, M., Xu, B., Zhu, C., Wang, S., Wang, P., Wang, X., & Mao, Z. (2026). *A-RAG: Scaling agentic retrieval-augmented generation via hierarchical retrieval interfaces*. arXiv. https://doi.org/10.48550/arXiv.2602.03442

Hierarchical agentic RAG. (2026, February 9). Emergent Mind. https://www.emergentmind.com/topics/hierarchical-agentic-rag

Hutto, C. J., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of the International AAAI Conference on Web and Social Media, 8*(1), 216–225. https://doi.org/10.1609/icwsm.v8i1.14550

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9475.

Pucek, B. (2026, March 12). LangGraph vs AutoGen vs CrewAI: Choosing the right agent framework for production. *The Thinking Company*.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 3982–3992). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1410

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv. https://doi.org/10.48550/arXiv.1910.01108

S., E., & Zhang, B. (2024, December 19). Building effective agents. *Anthropic*. https://www.anthropic.com/engineering/building-effective-agents

---

A Appendix A: System Architecture Diagram

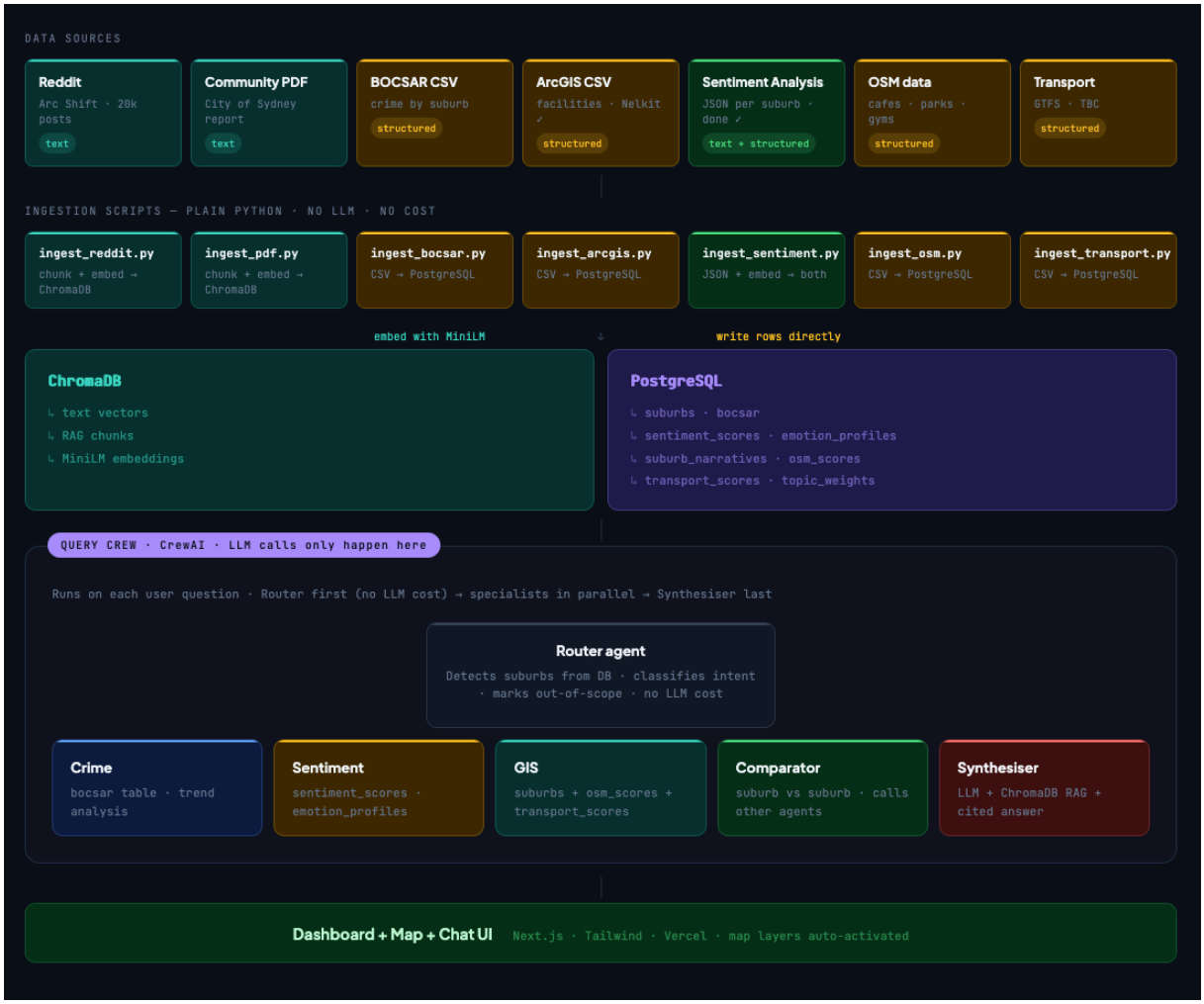

Figure 6: System architecture diagram.

B Appendix B: Dashboard Screenshots

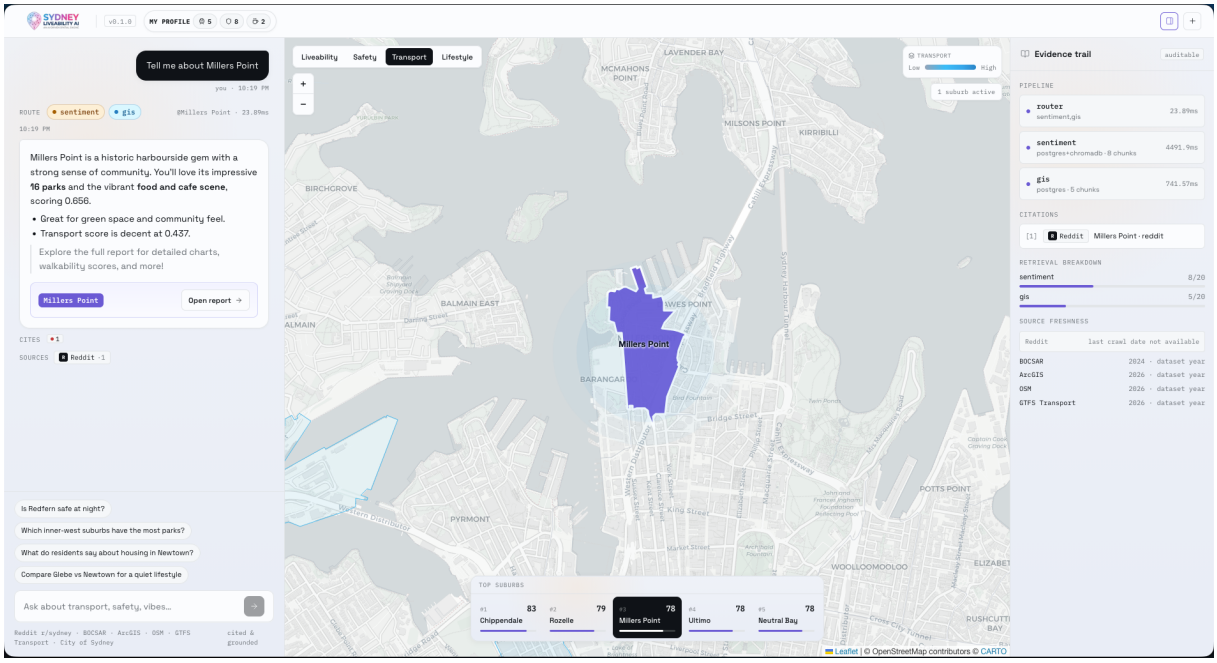

Figure 7: Leaflet map and chat interface.

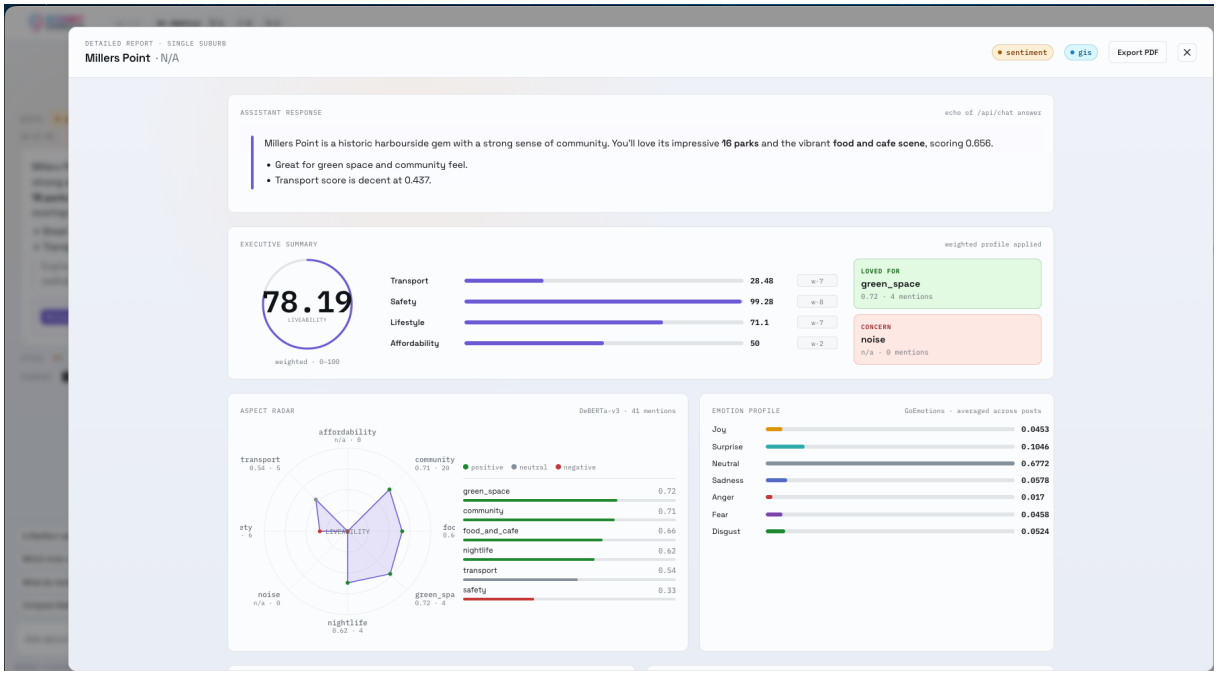

Figure 8: Detailed report with aspect radar and emotion profile.

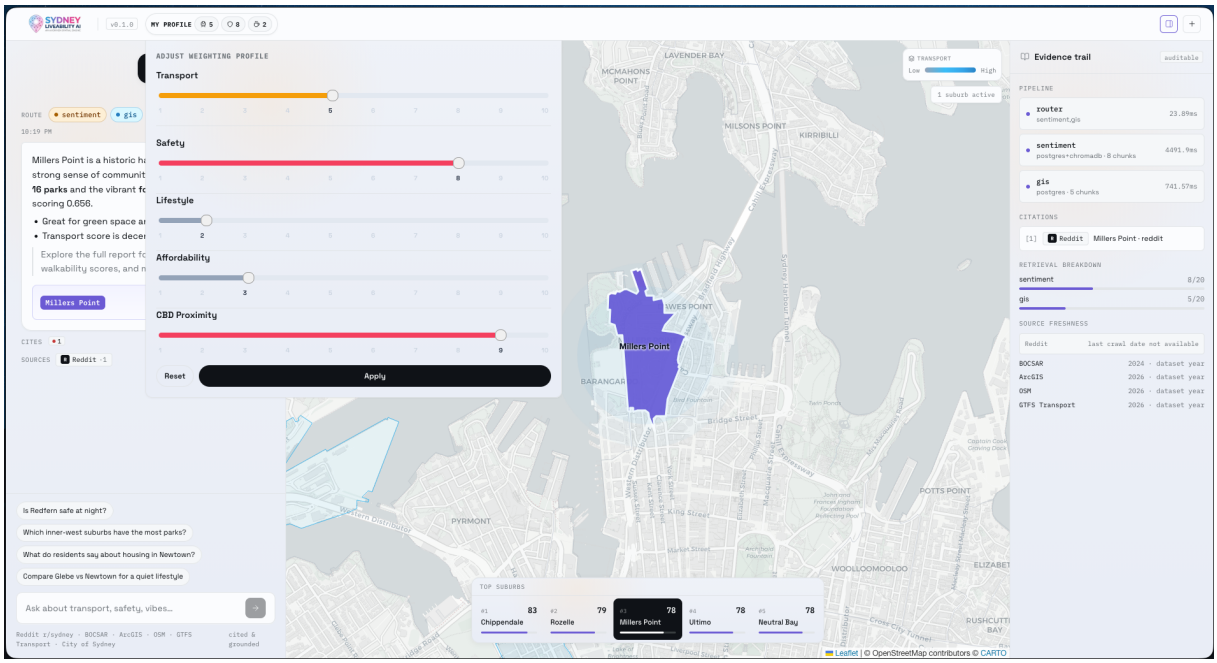

Figure 9: Weight adjustment panel.

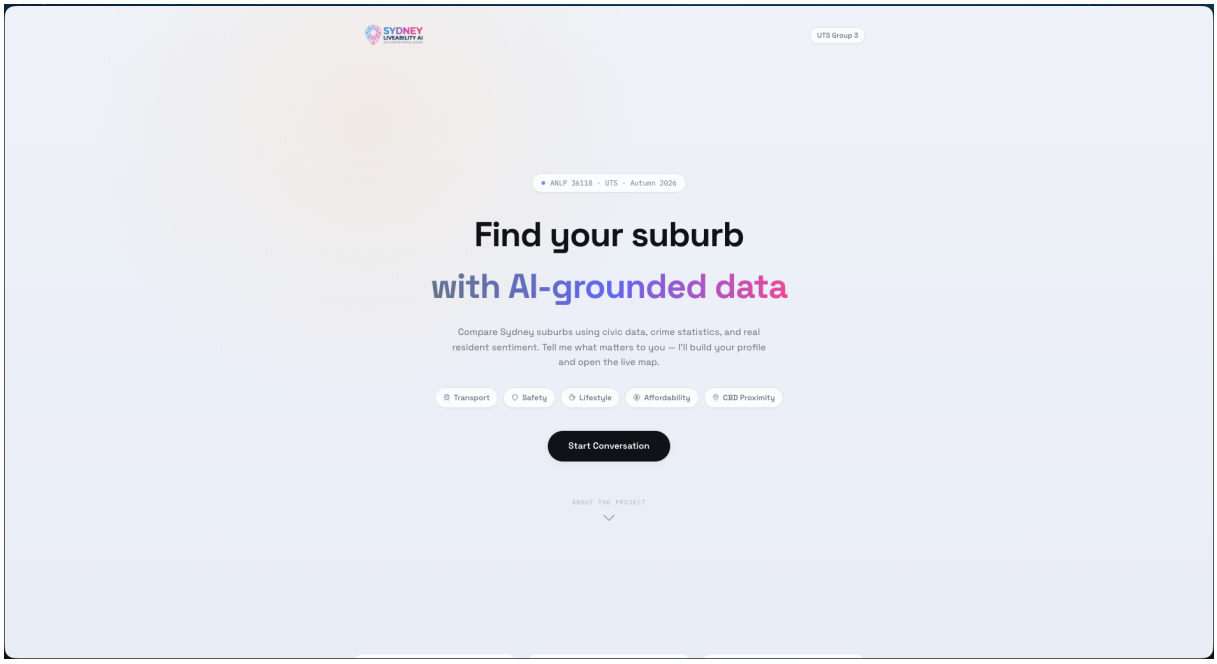

Figure 10: Landing page.

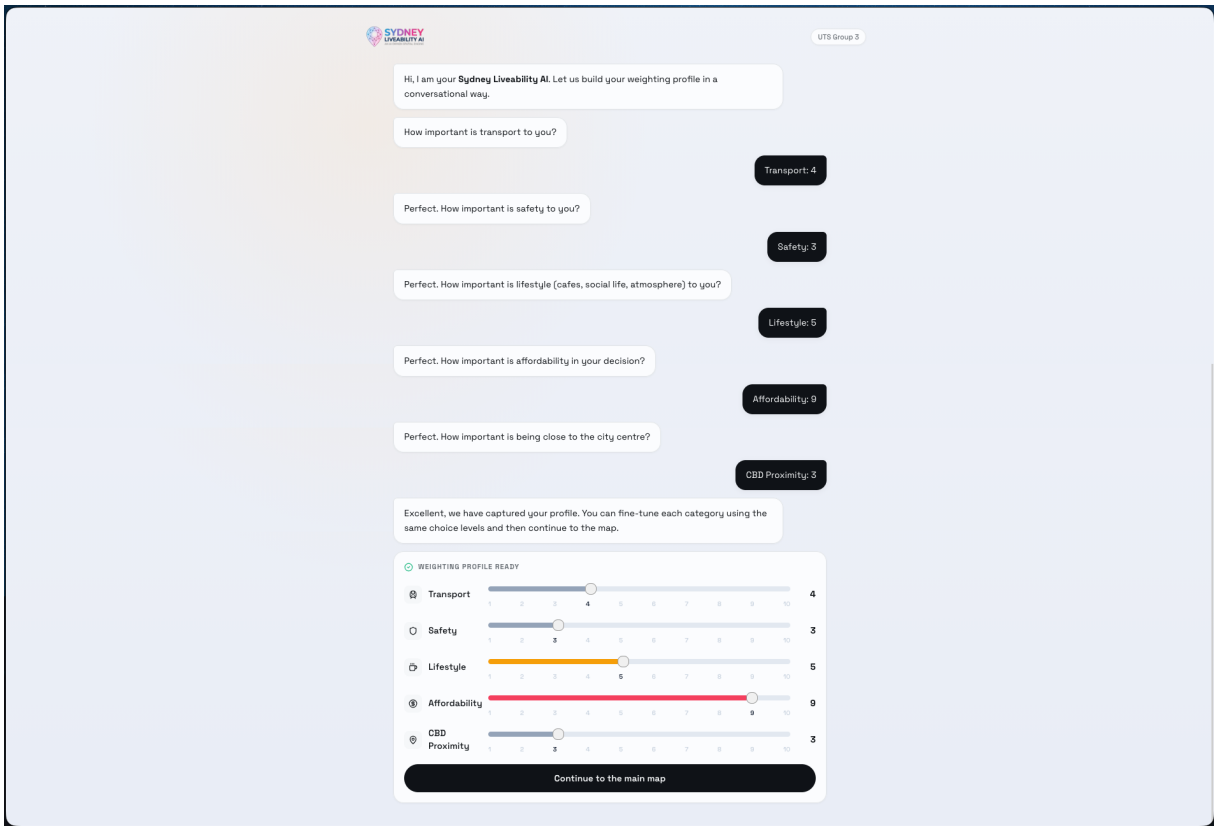

Figure 11: Onboarding page.

C Appendix C: Sample Code

C.1 ingest_sentiment.py (ingest pipeline)

File: ingest_sentiment.py

```
"""Ingest sentiment-pipeline outputs into ChromaDB.

Reads data/processed/reddit_analyses/{suburb}.json produced by the NLP
pipeline and upserts summaries/quotes into the embedding store.
"""
# ...imports...

def _records_for_narrative(suburb: str, narrative: str) -> list[dict]:
    if not narrative or len(narrative.strip()) < MIN_TEXT_LEN:
        return []
    chunks = chroma.chunk_reddit_text(narrative)
    post_id = _hash_id("narrative", suburb + narrative)
    return [
        {
            "text": chunk,
            "metadata": {
                "suburb": suburb,
                "source": "sentiment_narrative",
                "dimension": "general",
```

```
                "chunk_index": idx,
                "post_id": post_id,
            },
        }
        for idx, chunk in enumerate(chunks)
    ]

def _records_for_quote(suburb: str, quote: dict) -> list[dict]:
    text = (quote.get("text") or "").strip()
    if len(text) < MIN_TEXT_LEN:
        return []
    chunks = chroma.chunk_reddit_text(text)
    url = quote.get("url") or ""
    score = int(quote.get("score") or 0)
    post_id = _post_id_from_url(url) or _hash_id("quote", text)
    return [
        {
            "text": chunk,
            "metadata": {
                "suburb": suburb,
                "source": "sentiment_quote",
                "dimension": "general",
                "chunk_index": idx,
                "post_id": post_id,
                "post_score": score,
                "url": url,
            },
        }
        for idx, chunk in enumerate(chunks)
    ]

def ingest(input_dir=DEFAULT_INPUT_DIR, batch_size=64, suburb_limit=None):
    files = sorted(p for p in input_dir.glob("*.json") if not p.name.startswith("_"))
    buffer = []
    for path in files[:suburb_limit]:
        analysis = json.load(open(path, "r", encoding="utf-8"))
        suburb = analysis.get("suburb") or path.stem.replace("_", " ").title()
        buffer.extend(_records_for_narrative(suburb, analysis.get("narrative", "")))
        for quote in analysis.get("sources") or []:
            buffer.extend(_records_for_quote(suburb, quote))
        if len(buffer) >= batch_size:
            chroma.upsert_chunks(buffer); buffer.clear()
    if buffer:
        chroma.upsert_chunks(buffer)
```

C.2 CrewAI: query crew orchestration

File: query_crew.py

```
def build_query_crew() -> Crew:
    return Crew(
        agents=[
            router_agent,
```

```
            crime_agent,
            sentiment_agent,
            gis_agent,
            comparator_agent,
            synthesiser_agent,
        ],
        process=Process.sequential,
        verbose=True,
    )

def run_query(question: str, weights: dict | None = None) -> dict:
    router_output = run_router({"question": question})
    categories = list(router_output.get("categories", []))
    suburbs = list(router_output.get("suburbs_mentioned", []))

    if "out_of_scope" in categories:
        return run_synthesiser({"question": question, "router": router_output, "outputs": {}})

    specialist_outputs = {}
    for specialist_name in ["crime", "sentiment", "gis"]:
        if specialist_name in categories and suburbs:
            run_func = {"crime": run_crime, "sentiment": run_sentiment, "gis": run_gis}[
    specialist_name]
            specialist_outputs[specialist_name] = {}
            for suburb in suburbs:
                specialist_outputs[specialist_name][suburb] = run_func({"suburb": suburb, "
    question": question})
    # comparator, synthesiser, scoring, and final packaging follow...
```

C.3 CrewAI: sentiment agent (tool wrappers + trace)

File: sentiment.py

```
# Per-call trace accumulator via ContextVar
_TRACE_CTX: contextvars.ContextVar[Optional[list[dict]]] = contextvars.ContextVar("
    sentiment_trace_ctx", default=None)

@tool("get_suburb_aspect")
def _get_suburb_aspect_tool(suburb: str, dimension: str) -> dict:
    result = agent_tools.get_suburb_aspect(suburb=suburb, dimension=dimension)
    _record_call("get_suburb_aspect", {"suburb": suburb, "dimension": dimension}, result,
    elapsed_ms)
    return result

@tool("search_posts")
def _search_posts_tool(suburb: str, query: str, dimension: str = "", k: int = 5) -> dict:
    result = agent_tools.search_posts(suburb=suburb, dimension=dimension or None, query=query,
    k=k)
    _record_call("search_posts", {"suburb": suburb, "dimension": dimension, "query": query, "k"
    : k}, result, elapsed_ms)
    return result

# The agent uses these tools in a ReAct-style loop and exposes an evidence_trace
```

```
# that the crew/orchestrator can summarise for quality instrumentation.
```

C.4 RAG / Retrieval helpers (structured lookup + dense search)

File: tools.py

```
@lru_cache(maxsize=512)
def _load_cached_analysis(suburb: str) -> Optional[dict]:
    # loads SentimentScore, EmotionProfile, SuburbNarrative from Postgres
    # returns structured aspect entries and narrative/sources

def search_posts(suburb: str, dimension: Optional[str], query: str, k: int = 5) -> dict:
    # If the cached analysis marked (suburb,dimension) as null -> short-circuit no_data
    # Otherwise call db.chromadb.query_chunks(query=query, k=k, filters={"suburb": suburb, "
    dimension": dimension})
    hits = _query_chunks(query=query, k=k, filters=filters)
    return {"status": "ok", "suburb": suburb, "results": hits, ...}

def get_suburb_aspect(suburb: str, dimension: str) -> dict:
    # Return structured aspect entry or {"status":"no_data", ...}
```

Also see ingestion/upsert in `chroma.upsert_chunks(...)` invoked by ingest_sentiment.py for embedding-backed RAG.

C.5 FastAPI endpoints (chat + subreddit analysis)

Files: chat.py and reddit_router.py

```
# /api/chat
@router.post("/api/chat")
def chat(payload: ChatRequest) -> dict:
    question = (payload.question or payload.message or "").strip()
    if not question:
        return {"answer": "Please provide a question.", "sources": [], "suburb_scores": [], "
    map_state": None}
    response = run_query(question, payload.weights or {})
    return {
        "answer": response.get("answer"),
        "sources": response.get("sources", []),
        "suburb_scores": response.get("suburb_scores", []),
        "map_state": response.get("map_state"),
        # plus optional debug/quality fields when present
    }

# /api/reddit/{suburb} (on-demand pipeline + cache write)
@router.get("/api/reddit/{suburb}")
def analyse_suburb_endpoint(suburb: str) -> dict:
    normalised = _normalise_suburb(suburb)
    pg_cached = _pg_lookup(normalised)
    if pg_cached is not None:
```

```
        return pg_cached
    # fallback: run pipeline locally and persist
    posts = load_suburb_posts(normalised)
    result = analyse_suburb(normalised, posts)
    response = result.to_dict()
    _pg_write(response)
    _cache_write(_get_supabase(), response)
    return response
```

D Appendix D: Evaluation Tables and Charts

Include suburb-level scores per sentiment dimension, emotion distributions, transport scores, and POI density charts.

E Appendix E: Traditional vs. Modern NLP Comparison Table

Table 8: Traditional vs. Modern NLP approach comparison.

Approach	**Type**	**Used**	**Rationale**
VADER sentiment	Traditional	No	Lexicon-based; no aspect granularity.
LDA topic modelling	Traditional	No	Unsupervised; low coherence on Reddit text.
TF-IDF keyword extraction	Traditional	No	Frequency-based; no semantic understanding.
DistilRoBERTa aspect sentiment	Modern	Yes	8-dimension aspect scoring, contextual.
all-MiniLM-L6-v2 embeddings	Modern	Yes	Semantic similarity, compact and fast.
RAG using an LLM-agnostic model for response synthesis	Modern	Yes	Grounded, cited answers from retrieved chunks.
CrewAI multi-agent pipeline	Modern	Yes	Modular orchestration, scalable per suburb.